adgate

Ads are coming to AI. Nobody can check what happened.

adgate is the policy and verification layer between an AI product and its advertisers. It decides whether a sponsored slot belongs in a conversation, and **signs a record of every decision** that an advertiser can verify without trusting us.

- Pre-seed
- Research began 2 August 2026
- Working MVP, no users yet
- Source-available

Slide 02 · Problem

An assistant is the least auditable surface advertising has ever had

A banner sits in a page you can screenshot. An ad inside a conversation appears once, to one person, next to text no one else will ever see.

The advertiser can't check

No way to know whether their brand appeared beside a medical question, a bankruptcy, or a breakdown. They are asked to take it on faith.

The app can't prove it

Even a careful developer has no artefact showing the ad was chosen without reading the model's answer, or that sensitive turns were skipped.

The network is the referee

Every ad network grades its own homework. That was survivable on web pages. It will not be in a medium this intimate.

Slide 03 · Why now

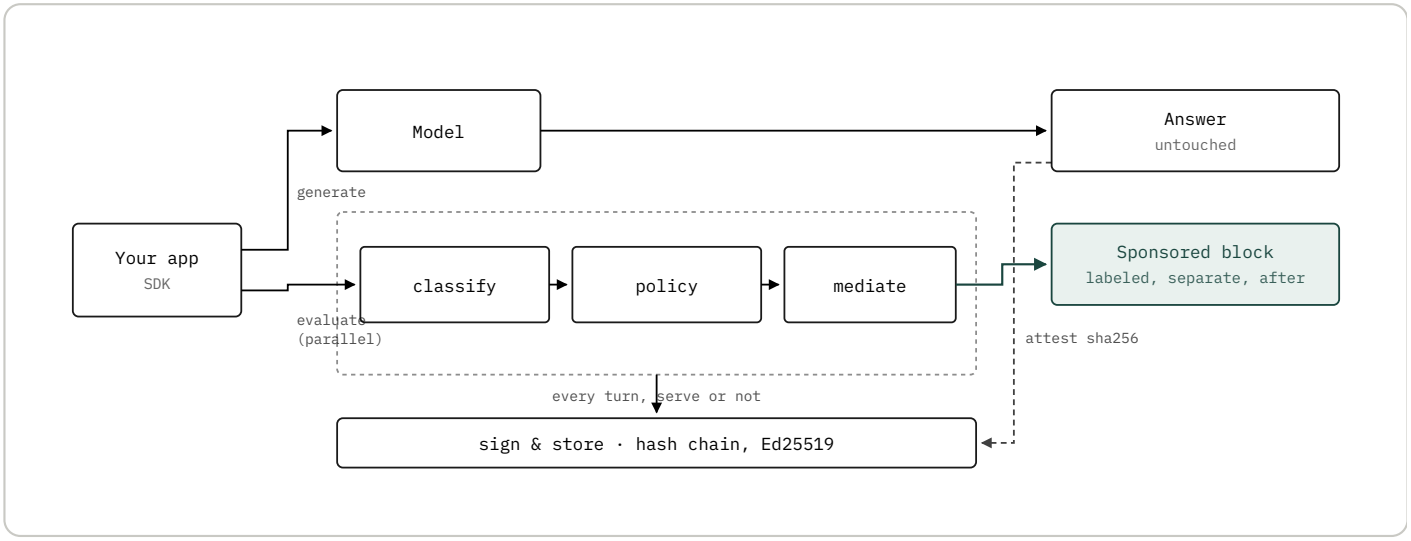
The surface is being monetised this year, and the guardrails do not exist

SUPPLY	**Thousands of AI apps have large free tiers and no way to pay for them.** Inference is a per-token cost with no per-user revenue. Every one of them is looking at advertising.
DEMAND	**Advertisers already buy developer attention** through newsletters and sponsorships, and understand sponsored placement. They do not yet have a way to buy it here safely.
REGULATION	**Disclosure rules are arriving for AI output regardless of advertising.** A signed record of what was shown and why is going to be asked for. Building it after the fact is far harder.

Slide 04 · Product

A gateway, not a network

The decision runs beside the model, not after it. The ad renders as its own block once the answer is finished. Only a hash of that answer ever comes back to us.



One turn. Both paths run at once and meet only in the rendered output, as two blocks that are never merged. A record is written whether or not an ad served, which is what makes a refusal auditable too.

Slide 05 · What it enforces

Nine outcomes. Eight of them are refusals.

serve | paid_user | region_blocked | sensitive_category | low_confidence | low_commercial_intent | frequency_cap | no_fill | error

Every refusal is signed and recorded exactly like a served ad. That is the part ad systems leave out, and it is the part an advertiser and a regulator both want.

Never inside the answer

The React component refuses to render inside model output, during server rendering too, so a wrongly nested ad is never even serialised.

Sensitive turns never reach demand

Nine categories blocked by default; self-harm cannot be removed. No advertiser is queried for a suppressed turn, so there is nothing to leak.

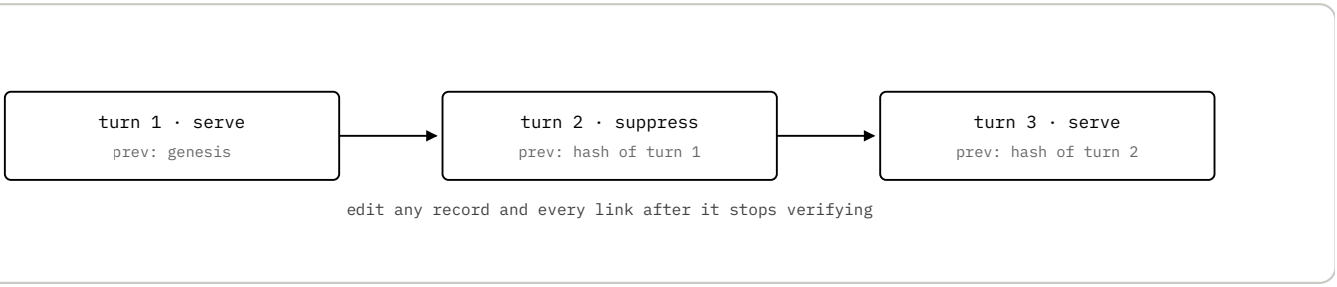
Failure is a refusal

Any timeout, outage or low confidence resolves to suppress. The host app shows no ad and keeps working. It never breaks because of us.

Slide 06 · The moat

Proof is the product

Each record hashes the one before it and is signed with Ed25519. Editing any record breaks every link after it. An advertiser verifies the chain offline, with a public key and no access to our database.



A downloadable report bundle re-verifies with the public key alone. This is what we sell, and what an ad network structurally cannot offer about itself.

Slide 07 · Adoption

Three edits, then it earns

SERVER	**Wrap the model.** Middleware starts the decision as generation begins, so we add no latency, and attests the finished answer.
BROWSER	**Render the slot after the answer.** A component that refuses to be nested inside model output.
CONFIG	**Paste your own affiliate ids.** Links are built with the app owner's identifiers at click time, so their commission stays theirs.
SELF-SERVE	**No sales call anywhere in that path.** Sign up, create an app, copy a key, paste a snippet. The console tells them why a creative cannot serve and what to change.

Slide 08 · Business model

We take a share, and we can prove we didn't cheat to get it

The usual objection to a mediator taking a cut is that it creates an incentive to serve more ads. Every network has that conflict and asks you to trust them. **We are the only one that can prove we did not act on it.**

Revenue share on served ads

The volume business, once apps are live. The open policy engine and the signed chain mean an advertiser can verify we did not quietly loosen a threshold to earn more.

Verification reports

Sold to advertisers. Highest margin, arrives later, and the reason a brand is willing to spend on this surface at all.

Nearer term: affiliate inventory needs no advertiser relationship, so an app can earn on day one and we can measure real fill rates before selling anything.

Slide 09 · Pricing

Three prices, arriving in the order they can be earned

FREE	**Self-hosted, and hosted below a usage threshold.** The gateway is source-available and runs on one service plus Postgres. Free removes every reason not to try it, and the hosted free tier is how we see real traffic.
REVENUE SHARE	**A percentage of ads we mediate, in the 15–30% band the market already understands.** Charged only on served ads, never on refusals, so we are never paid more for loosening a policy. Affiliate revenue an app sources itself stays entirely theirs.
VERIFICATION	**Per-report or annual, sold to the advertiser rather than the app.** The artefact a brand needs before spending on this surface. Highest margin and the least commoditisable of the three.

A hosted classifier is the one real cost per turn, which is why the free tier is bounded by usage rather than by feature.

Slide 10 · Unit economics

The model is simple. The inputs are exactly what we don't know yet.

Revenue per thousand conversations is the product of four numbers. We can measure three of them within weeks of launch, and the fourth is what an advertiser will pay.

ELIGIBLE RATE	**— to be measured** Share of turns that survive policy and reach demand. Our synthetic fixture run gave 23%, but that set is deliberately adversarial and is not a traffic estimate.
FILL RATE	**— to be measured** Share of eligible turns with matching inventory. Directly improvable by stocking categories, which is the one lever fully under our control.
CLICK-THROUGH	**— to be measured** The genuine unknown. A relevant, labelled recommendation after a helpful answer should outperform display, but nobody has published honest numbers for this surface.
PAYOUT PER CLICK	**— market rate** Developer-tool affiliate and sponsorship rates are public and knowable, so this is the least uncertain input.

> **We are not presenting a projection.** Any revenue-per-thousand figure we put on a slide today would be invented, and the first honest question would expose it. The instrumentation to measure all four already ships in the product; the plan below is designed to produce them.

Slide 11 · Go to market

Integrators first. Advertisers only once there are numbers.

Advertisers will not talk seriously without traffic. Affiliate inventory means apps can be paid on day one without a single advertiser existing, which breaks the usual chicken and egg.

1. Open-source chat frontends

Very large self-hosted user bases, no monetisation, and the same source-available posture. A working plugin is distribution, not a sales call.

2. Indie AI apps with free tiers

Their free tier is a cost centre they already worry about. Reached through the AI SDK community and recent launch channels.

3. Developer-tool advertisers

Companies already buying developer attention through sponsorships. Approached last, with real numbers and the verification report as the reason to trust the channel.

The message leads with the guardrails, not the revenue. Most AI founders react badly to "put ads in your product" and well to "ads that can never appear inside your model's answer, and that skip anyone in distress".

Slide 12 · On-chain

Anchoring makes the record trustless, not just verifiable

Today an advertiser verifies a signed chain, which still means trusting that we did not rewrite history wholesale. Publishing periodic Merkle roots on-chain removes that last assumption.

PUBLISHED	A Merkle root per app per interval. Nothing else.
NEVER PUBLISHED	Conversation text, user identifiers, creative content, advertiser delivery data. The chain carries hashes, not payloads.
RESULT	An advertiser checks a record against a root nobody can retroactively edit, including us. Verification stops depending on our good behaviour.

> **Status: designed, not built.** The hash chain and the signing exist today and produce exactly the roots this needs. The anchoring itself is the next build, not a shipped feature.

Slide 13 · Competition

Everyone strong here is strong at the thing we deliberately don't do

AI AD NETWORKS	**Koah, Gravity and similar.** They own demand, which is real strength. But a network auditing itself is not a claim anyone will accept for long. We integrate them as a demand source rather than competing on inventory.
INCUMBENT NETWORKS	**Could extend downward.** Their model is profiling and auction depth. Both are liabilities in a conversation, and neither is something they can credibly give up.
THE MODEL PROVIDERS	**The real threat.** They can put ads in their own surface and keep everything. But they cannot be the neutral referee for the thousands of apps built on top of them, and those apps are the market we serve.
WHY US	We own no inventory, take no cut of an app's affiliate revenue, and store no conversations. That neutrality is not a marketing line, it is checkable in the source and in the chain.

Slide 14 · Status

A working MVP, and no users. Both of those are true.

40	2,291	83	0	0
STORIES SHIPPED	TESTS PASSING	REVIEW FINDINGS FIXED	USERS	REVENUE

What runs today

Gateway, policy engine, two-stage classifier, demand mediation, signed audit chain, verification endpoints, TypeScript and Python SDKs, a React component, a developer console with self-serve signup, and two example apps. Green CI.

What does not

Nothing is deployed publicly. No real advertiser inventory, only templates. No revenue, no users beyond my own test traffic. On-chain anchoring is designed, not built.

> **Researched from 2 August 2026.** The idea was worked out before it was built, and the implementation itself is young. Five adversarial review passes over that code produced 83 findings, all fixed, including a fail-open path in the classifier and a chain check that would have accepted a forged record.

Slide **15** · Next

Prove the surface works before selling anything on it

WEEKS 1–2	**Deploy, publish the SDK, stock real affiliate inventory.** Nothing can be measured until a developer can install a package and see fill.
WEEKS 3–8	**Three to five apps integrated.** Open-source chat frontends first: large free user bases, no monetisation, and the same licence posture.
WEEKS 8–12	**Take real numbers to advertisers.** Eligible-turn rate, fill rate, click-through. Sell a fixed-price pilot with the verification report included.
THE OPEN QUESTION	Whether eligible-turn rates and click-through clear the economics at all. The affiliate step exists to find that out cheaply, before building billing for a market that might not clear.